Scope and Closures

Lecture 1 Where a function finds its names

Gregory S. DeLozier, PhD

A function that remembers

```
function makeAdder(a) {
    return function(b) { return a + b };
};
addOne = makeAdder(1);
print(addOne(2));
```

What must survive after makeAdder returns?

Dynamic scope and lexical scope

Dynamic scope follows the caller’s environment.

Lexical scope follows the defining environment.

Static scope is another name for lexical scope.

Which greeting?

```
greeting = "Hello";
function showGreeting() { print(greeting) };
function frenchCaller() {
    greeting = "Bonjour";
    showGreeting();
};
frenchCaller();
```

The parent link decides

```
# Dynamic scope
call_environment = {PARENT_KEY: environment}

# Lexical scope
call_environment = {
    PARENT_KEY: callee["environment"]
}
```

A closure

```
function(b) { return a + b }
```

Function code + defining environment

b is a parameter of the inner function. a is free in the inner function.

The closure supplies the outer binding of a.

Tracing makeAdder(1)

1. Create a factory-call environment with a = 1.
2. Evaluate the inner function expression.
3. Retain that environment in the function value.
4. Return the function.

Tracing addOne(2)

```
addOne call environment
    b = 2
    $PARENT: retained makeAdder environment

retained makeAdder environment
    a = 1
    $PARENT: top-level environment
```

Lookup b stops locally. Lookup a follows one parent link.

Separate factory calls

```
addOne = makeAdder(1);
addTen = makeAdder(10);
print(addOne(5));
print(addTen(5));
print(addOne(20));
```

Same function body. Different retained environments.

Shadowing

```
x = 100;
function makeReader(x) {
    return function() { return x };
};
readSeven = makeReader(7);
print(readSeven());
print(x);
```

Shadowing leaves the outer binding intact

```
x = 100;
function inspect(x) {
    print(x);
};
inspect(7);
print(x);
```

Prints 7, then 100

The lookup stops at the first binding

Environment	Contains x?	Lookup action
Returned reader’s call frame	No	Follow $PARENT
Retained makeReader frame	Yes: x = 7	Return 7 and stop
Top-level environment	Yes: x = 100	Never reached

Local assignment can introduce shadowing

```
x = 10;
function f() {
    print(x);
    x = 20;
    print(x);
};
f();
print(x);
```

Prints 10, then 20, then 10

Arguments and body use different environments

```
x = 3;
function addX(n) { return x + n };
function caller() {
    x = 20;
    return addX(x);
};
print(caller());
```

Capturing preserves a reference

```
amount = 1;
function currentAmount() { return amount };
amount = 9;
print(currentAmount());
```

The environment stays the same. Its binding changes.

Function creation in the evaluator

```
return {FUNCTION_KEY: function_count,
        "parameters": ast["parameters"],
        "body": ast["body"],
        "environment": environment}, None
```

Recursion and reachability

```
function factorial(n) {
    if (n == 0) { return 1 };
    return n * factorial(n - 1);
};
print(factorial(6));
```

Every recursive call gets a fresh n.

Practice

Write makeMultiplier(factor). Return a function that multiplies its argument by factor.

Create double and triple. Explain which bindings each call creates.

Questions for the next lecture

How can two functions share private state?

What should assignment do to an outer binding?

What makes a group of closures act like an object?